

BI theme digest agent

An agent that turns a week of client calls and support cases into a themed digest where every sentence traces back to the second it was said.

● Jim Mehta

2026-09-16

Built for Momentive Software. Two public repositories: the code, and the context store it writes to.

The brief, restated

Build an agent that reads what clients are telling us across calls and support cases, and turns it into something a product team can act on every week. Three things had to be true, and each one drove a design decision rather than a paragraph.

- 1 **Every claim traceable to its moment.** A sentence reaches the digest only if it carries a claim id, and a claim exists only if its quoted text is an exact substring of the source span it cites. Code asserts that substring, not the model.
- 2 **Context that persists between runs.** The themes live in their own repository and are appended to rather than regenerated, so week three costs less than week one and the store is the memory.
- 3 **An agent whose access is bounded.** The agent that holds the most context in the system has four read-only tools, and no write tool exists for it to call.

55

claims verified across three corpus weeks

0

claims rejected, and all five rejection reasons sit at zero

55 of 55

citations resolved to real source text, checked twice

Those three figures come out of the committed audit logs and the golden set run, not out of a summary I wrote. Every number in this deck is traceable to a manifest field, and where a measurement is missing the slide says so.

What it does

Nine stages. Three model agents run the two that need judgment, two readers at the extraction tier and the editor at the synthesis tier. Code runs six, and a person runs the ninth, which is the division of labour the whole design rests on.



Nightly. The connectors pull the previous day, the scrubber removes personal data before any model sees a document, two reader agents at the extraction tier pull claims, and code validates the schema and verifies every citation.

Weekly. Code enriches accounts deterministically, the editor agent at the synthesis tier reads the theme index first and decides append or open new, code scores the themes, and code performs every write.

Then a person. Nothing reaches a system of record until a human runs approve with an explicit confirmation.

Tier, not model. Application code asks for a tier. `config/models.yaml` is the only runtime file that names a model, and a model id under the source tree fails the build.

The two sources

Gong for calls and Salesforce for support cases, behind two local servers that advertise seven tools between them. In production those same seven names are backed by the real APIs.

Gong

Call metadata by date range, the participant list that maps a speaker id to a person, and the transcripts.

Salesforce

SOQL through the REST query endpoint, over Case, CaseComment, Account and User.

Five rules are enforced at the tool boundary, in the tool, not in a prompt.

- 1 There is no write tool to call, not a disabled one and not a gated one.
- 2 The date window comes from the launcher, and a request outside it is refused rather than trimmed.
- 3 IsPublished equals true sits in the SOQL itself, so a private comment is never in the result set.
- 4 A field allowlist, so a column outside it cannot be asked for.
- 5 A row cap of 200, so a runaway loop cannot pull the corpus.

The agent cannot ask for what it is not allowed to see, because the tool will not form the query.

```
def sfdc_get_case(case_id: str) -> types.CallToolResult:
    entry = CORPUS.entry_for("sfdc_case", case_id)
    if entry is None:
        raise store.ToolError("not_found: no case with Id %s" % case_id)
    refusal = store.check_record_window("sfdc_get_case", [entry], CORPUS)
    if refusal is not None:
        return refusal
    case_file = CORPUS.file_for(entry, "SalesforceCaseFile")
    all_comments = case_file["comments"]
    # Rule 3. The private rows are read only to be counted; their bodies never enter
    # any structure that is serialised into the response.
    published = [c for c in all_comments if c.get("IsPublished") is True]
    withheld = len(all_comments) - len(published)
    published.sort(key=lambda c: (c["CreatedDate"], c["Id"]))
    page = published[:CONFIG.row_cap]
    comments = [store.project(c, store.SFDC_COMMENT_FIELDS, keep_attributes=True) for c in page]
    soql = (
        "SELECT %s FROM CaseComment WHERE ParentId = '%s' AND IsPublished = true "
        "ORDER BY CreatedDate ASC LIMIT %d"
        % ("", ".join(store.SFDC_COMMENT_FIELDS), case_id, CONFIG.row_cap)
    )
    document = {
        "schema_version": store.SCHEMA_VERSION,
        "case": store.project(case_file["case"], store.SFDC_CASE_FIELDS, keep_attributes=True),
        "comments": comments,
        "meta": store.meta(CONFIG, len(comments), store.SFDC_COMMENT_FIELDS,
                           withheld_comment_count=withheld, soql=soql),
    }
    return store.ok(document, "SalesforceCaseResponse")
```

mcp/salesforce_server.py

mcp/salesforce_server.py

One claim, traced end to end

One sentence in the digest, followed all the way back to the second it was said. Four steps, one lookup each, real values throughout.

1 **The digest sentence**

Great Lakes Museum Alliance says every renewal statement it sends is missing the balance carried over from the previous period, so the invoice total reads far higher than it should.

../bi-theme-digest-store/digests/
2026-W37.md

2 **The evidence line under it**

claim 03a7ca4ae6d8 | call 7782934451002 at 11:03 | Rhonda Calloway | Great Lakes Museum Alliance (customer)

../bi-theme-digest-store/digests/
2026-W37.md, theme 1 evidence

3 **The claim record in the store**

{"claim_id": "03a7ca4ae6d8", "source_id": "7782934451002", "speaker_id": 4521, "window_ms": [663288, 722760]}

../bi-theme-digest-store/evidence/
claims/2026-09-08T06:00Z.jsonl

4 **The transcript turn it cites**

At 681342 ms, inside that window: "Every renewal statement we send out is missing the balance that carried over from the previous period, so the invoice total reads far higher than it should." An exact substring, asserted by code. 663288 ms is the 11:03 the digest printed.

data/mock/traps/gong/calls/
7782934451002.json

All 55 claims and all 125 digest evidence lines resolve this way. A claim whose citation does not resolve is rejected and logged, never patched.

The context store

A separate repository with no code in it, and the pipeline's only write credential reaches it and nothing else.

- 0

Layer 0, the router file. Always read, 4351 bytes today, says what is here and in what order to read it.
- 1

Layer 1, the theme index. Always read. One line per theme: id, title, product area, status, score, counts, last run, aliases.
- 2

Layer 2, one theme file. Opened only when the index says that theme is relevant. Reading the whole directory is a mistake, not a shortcut.

Progressive disclosure is also least privilege here, because layer 2 can be gated by role.

Every commit is made by pipeline code after that code validated a document against a schema, so git log here is an audit trail rather than a changelog. 26 commits, and every one of them is a pipeline run.

10 / 0

W37, opened and appended

1 / 7

W38, opened and appended

0 / 8

W39, opened and appended

```
themes/_INDEX.md, top 5 of 11 rows by score

THEME-0011 score 82 open integrations
Role changes are not propagated by the user sync, so people keep access they should have lost
aliases: identity provider sync; provisioning sync; role change; permission set

THEME-0002 score 81 open accounting
Renewal amounts ignore prior-period credits and mid-year upgrades, so members are over-billed
aliases: renewal statement; renewal invoice; partial year adjustment credit; mid-year upgrade

THEME-0001 score 75 open reporting
Report exports stop partway through above a row threshold, with no warning that data is missing
aliases: membership export; report export; year end donor report; data export

THEME-0006 score 75 open events
Event check-in cannot run without network and sync afterwards
aliases: event check-in; attendee kiosk

THEME-0003 score 70 open fundraising
Pledge reminders send on the standard schedule and channel regardless of donor preferences
aliases: pledge reminders; single reminder; mail only donors; paper only

../bi-theme-digest-store/themes/_INDEX.md

../bi-theme-digest-store/themes/_INDEX.md

38204e1 run 2026-09-28T08:00Z: ask, answered
c237d18 run 2026-09-28T09:00Z: readie author name corrected
d4e5397 run 2026-09-28T09:00Z: approve dry run, nothing filed
65b7b0a run 2026-09-28T08:00Z: ask, declined
212fc38 run 2026-09-28T08:00Z: ask, answered
bf59c93 run 2026-09-14T07:00Z: swap 2026-W37 onto models.cheap.yaml, jaccard 1.000
e93f5ba run 2026-09-14T07:00Z: stability for 2026-W37, jaccard 1.000
05ec3a2 run 2026-09-28T07:00Z: build 2026-W39 digest, 8 themes appended, 0 opened
99b01b1 run 2026-09-21T07:00Z: build 2026-W38 digest, 7 themes appended, 1 opened
c51b740 run 2026-09-14T07:00Z: build 2026-W37 digest, 0 themes appended, 10 opened
ebdd772 run 2026-09-25T06:00Z: ingest 2026-09-25, 5 sources, 5 claims
8c33228 run 2026-09-24T06:00Z: ingest 2026-09-24, 1 sources, 1 claims
cf44727 run 2026-09-23T06:00Z: ingest 2026-09-23, 3 sources, 3 claims
5582b25 run 2026-09-22T06:00Z: ingest 2026-09-22, 2 sources, 2 claims
7f75cee run 2026-09-21T06:00Z: ingest 2026-09-21, 1 sources, 1 claims
57df964 run 2026-09-18T06:00Z: ingest 2026-09-18, 2 sources, 2 claims
578d23f run 2026-09-17T06:00Z: ingest 2026-09-17, 4 sources, 4 claims
e772967 run 2026-09-16T06:00Z: ingest 2026-09-16, 4 sources, 4 claims
0cc12d1 run 2026-09-15T06:00Z: ingest 2026-09-15, 3 sources, 3 claims
44428f6 run 2026-09-14T06:00Z: ingest 2026-09-14, 3 sources, 3 claims
7919d63 run 2026-09-11T06:00Z: ingest 2026-09-11, 4 sources, 4 claims
0f1a3f5 run 2026-09-10T06:00Z: ingest 2026-09-10, 5 sources, 5 claims
dcf1a80 run 2026-09-09T06:00Z: ingest 2026-09-09, 7 sources, 7 claims
ccff89 run 2026-09-08T06:00Z: ingest 2026-09-08, 9 sources, 11 claims
896dde5 run 2026-09-07T06:00Z: ingest 2026-09-07, 0 sources, 0 claims
8d84584 scaffold: empty context store, ROUTER.md and the two index files

$ git -C ../bi-theme-digest-store log --oneline

$ git -C ../bi-theme-digest-store log --oneline
```

Two customers, one theme

Trap T1 in the mock corpus: two customers describe the same billing failure in vocabulary that shares almost no words, and they have to land on one theme.

Great Lakes Museum Alliance, on a call

"Every renewal statement we send out is missing the balance that carried over from the previous period, so the invoice total reads far higher than it should."

Prairie Land Trust Council, in a support case

"Members who upgraded partway through the year are billed the whole annual figure again with nothing knocked off, and we end up correcting each invoice by hand."

Both attached to THEME-0002, and the editor stated its reason.

"In all three, what the member already paid or is owed never reaches the amount billed. Two customers are sending members numbers they then have to apologise for."

T1a: Great Lakes Museum Alliance, gong
"There is a problem in the output that we have been working around since the spring though, and the board has now noticed it. Every renewal statement we send out is missing the balance that carried over from the previous period, so the invoice total reads far higher than it should."

T1b: Prairie Land Trust Council, salesforce
"Members who upgraded partway through the year are billed the whole annual figure again with nothing knocked off, and we end up correcting each invoice by hand."

THEME-0002: Renewal amounts ignore prior-period credits and mid-year upgrades, so members are over-billed
editor's stated reason: Prairie Land Trust Council says that when one of its chapters upgrades partway through the year, the fall renewal bills the full amount again with nothing knocked off for what was already collected. That is a customer paying twice for the same period until someone notices.

../bi-theme-digest-store/themes/THEME-0002.md

../bi-theme-digest-store/themes/THEME-0002.md, evidence and rationale

The assertion that checks it: the two T1 claims, worded completely differently, are attached to exactly one theme between them.

It passes, on claims 03a7ca4ae6d8 and 306b8bbab997.

evals/golden_set.yaml, assertion S2

The same feature under two names

Trap T2: one customer says event check-in and another says attendee kiosk, and they mean the same capability.

Cascadia Nurses Association

"When the venue wifi drops we need event check-in to keep running on the tablets and sync everything back up once we are online again."

Atlantic Shipwrights Guild

"If the hall wifi cuts out halfway through the morning, the attendee kiosk has to carry on signing people in and catch up later."

```
title: Event check-in cannot run without network and sync afterwards
aliases: [event check-in, attendee kiosk]
product_area: events
status: open
created_run: 2026-09-14T07:00Z
last_updated_run: 2026-09-28T07:00Z
accounts: [ACC-0002, ACC-0004]
```

```
../bi-theme-digest-store/themes/THEME-0006.md
```

```
../bi-theme-digest-store/themes/THEME-0006.md
```

Aliases are a column in the theme index, which is the first thing the editor reads, so the merge is context rather than a decision made again every Monday.

Both landed on THEME-0006, and the theme carries an aliases field reading event check-in and attendee kiosk, so the reconciliation is visible rather than implied.

This is the post-acquisition vocabulary problem, named once. A company that grew by acquisition carries several names for the same capability, and customers use whichever name their own product taught them. The aliases field is the cheap, auditable answer: next week's claim matches on either name, and a reviewer can see which names were merged and disagree with the merge.

What the agent refused to surface

Three things the corpus plants that must not come out the other end, and one that must come out ranked honestly.

A private case comment

A Salesforce comment saying the account is a churn risk. It was never in the result set, because `IsPublished equals true` is in the SOQL itself rather than in a filter afterwards. 10 private comments withheld across the three weeks, 4 of them in week 37, and the count is in the audit log.

```
{
  "schema_version": "1.0.0",
  "ts": "2026-09-10T07:07:43.649Z",
  "run_id": "2026-09-10T06:00Z",
  "agent": "pipeline",
  "action": "read",
  "stage": "connect",
  "target": "500800002a0pLr0AK",
  "model_tier": null,
  "model_id": null,
  "prompt_hash": null,
  "tokens_in": 0,
  "tokens_out": 0,
  "cache_read": 0,
  "cache_write": 0,
  "cost_usd": 0.0,
  "outcome": "ok",
  "detail": {
    "source": "salesforce",
    "doc_type": "case",
    "occurred_at": "2026-09-10T13:58:41.000Z",
    "window_from": "2026-09-10",
    "window_to": "2026-09-10"
  }
}
```

```
$ grep withhold runs/2026-09-10T06:00Z/run.log.jsonl | python -m json.tool
```

runs/2026-09-10T06:00Z/run.log.jsonl, withhold event

Personal data, before the call

5 values redacted before any model call: 1 email address, 1 phone number, 1 postal address and 2 personal names. The scrubber runs before the document is stored and before any model sees it, and the log records counts by kind and never the value.

```
{
  "schema_version": "1.0.0",
  "ts": "2026-09-10T07:07:42.777Z",
  "run_id": "2026-09-09T06:00Z",
  "agent": "scrubber",
  "action": "scrub",
  "stage": "scrub",
  "target": "7782934451404",
  "model_tier": null,
  "model_id": null,
  "prompt_hash": null,
  "tokens_in": 0,
  "tokens_out": 0,
  "cache_read": 0,
  "cache_write": 0,
  "cost_usd": 0.0,
  "outcome": "ok",
  "detail": {
    "EMAIL": 1,
    "PHONE": 1,
    "ADDRESS": 1,
    "NAME": 1
  }
}
```

```
$ grep scrub runs/2026-09-09T06:00Z/run.log.jsonl | python -m json.tool
```

runs/2026-09-09T06:00Z/run.log.jsonl, scrub event

An employee speaking for a client

A Momentive Software account executive saying a lot of our customers ask for this. Speaker side is derived by code from the connector's participant record, so a staff quote is rejected at the verifier, where it is a join rather than a hope.

Trap T3 is the opposite case. A prospect's request is surfaced rather than suppressed, and ranked below every customer backed theme: THEME-0009 sits last in the week on a score of 6, and the eval asserts that it still appears in the digest.

Four planted personal values, four greps over the whole store repository, four zero-hit results, all four in the eval.

Prioritization

The score is code. The narrative is the model. Both are printed on the page, so a reader can recompute one and argue with the other.

```
score = 30 * min(customers, 4) / 4 + 25 * min(ARR, 500000) / 500000 + 20 * min(open cases, 4) / 4 + 15 * max(0, 14 - days since last evidence) / 14 + 10 * min(claims, 3) / 3
```

Rounding is floor(total + 0.5), clamped to the range 0 to 100, because banker's rounding lets two correct implementations disagree on a half value.

src/digest/score.py

```
### 1. Renewal amounts ignore prior-period credits and mid-year upgrades, so members are over-billed (score 80)
```

```
Why this score: customers 30 * min(2, 4) / 4 = 15.00 + value 25 * min(494000, 500000) / 500000 = 24.70 + cases 20 * min(9, 4) / 4 = 20.00 + recency 15 * max(0, 14 - 4) / 14 = 10.71 + severity 10 * min(3, 3) / 3 = 10.00 -> 80.41 -> floor(+0.5), clamped: 80.
```

```
../bi-theme-digest-store/digests/2026-W37.md
```

```
../bi-theme-digest-store/digests/2026-W37.md, theme 1 score line
```

What the model wrote beside it

"Two customers are sending members numbers they then have to apologise for."

THEME-0002 worked: $15.00 + 24.70 + 20.00 + 10.71 + 10.00 = 80.41$, which rounds to 80. One of those two lines is checkable with a calculator. The other is the reason anybody reads the digest at all.

The digest

What a product manager actually reads. It is a file and a page rather than a service, because that is what a product team opens.

```
# Theme digest, week 2026-W37
Run 2026-09-14T07:00Z, week 2026-W37: 25 sources read (14 calls, 11 cases), 4 private comments withheld, 5
  PII redactions, 27 claims verified, 0 rejected, 0 themes appended, 10 opened, cost $2.27 for the week
  ($1.22 this build), model tiers used: extraction, synthesis.
## Cold start: this week's claims resolve into ten themes, six of them carrying evidence from two separate
  customers.
The store started empty, so every claim opened a theme; the work was grouping, not triage. Six themes are
  backed by two customers each: truncated exports, wrong renewal amounts, pledge reminders that ignore donor
  preferences, renewal email landing in spam, duplicate entries from the accounting sync, and offline event
  check-in. Three of those six were only visible after reconciling different vocabularies, most notably one
  account's 'attendee kiosk' and another's 'event check-in', and Great Lakes' missing carried-over balance
  sitting on the same billing failure as Prairie Land's un-prorated mid-year upgrades. Four themes rest on a
  single voice: job posting expiry, tier migration tracking, an LMS SCORM import request from a prospect
  rather than a customer, and one piece of unprompted praise for the fundraising platform. Nothing is quiet
  or stale yet because nothing has history.
```

```
../bi-theme-digest-store/digests/2026-W37.md
```

```
../bi-theme-digest-store/digests/2026-W37.md
```

The run line reports the week as work done

25 sources read, 14 calls and 11 cases. 4 private comments withheld. 5 personal data redactions. 27 claims verified, 0 rejected. 0 themes appended, 10 opened. 2.27 US dollars for the week.

Each theme carries a paragraph in plain sentences, then the evidence, each quote tagged with the account, the call or case, the timestamp, the speaker and the claim id.

A thin week is visible on the page a product manager already reads. That is the failure I care most about: not an error, a digest that looks completely normal and is quietly missing a day of calls.

Governance

Seven controls, each with the command that demonstrates it, because a governance claim that cannot be run is a paragraph rather than a control.

- 1 Personal data removed before the model.** 53 scrub events, one per source document. A grep for a planted value returns nothing, anywhere in the store.
- 2 Every action audited, reads included.** 1136 events across the store, 575 of them reads and 218 writes, one validated line each.
- 3 Private comments never in the result set.** 10 withhold events across the 15 ingest runs.
- 4 Structured output rejected, never patched.** 55 accepted, the rejected directory empty, all five rejection reasons at zero.
- 5 Every claim resolves to its source.** The eval prints 55 citations resolved and exits 0.
- 6 Least privilege at the tool boundary.** 4 read tools on the editor. A grep for a write tool matches nothing.
- 7 A human gate before any system of record.** The dry run prints the issue and files nothing. The same command without the confirmation exits 4, refused.

```
3:There is no write tool in this module, and there is no write tool anywhere else that the
33:TOOL_NAMES = ("read_theme_index", "read_theme", "list_run_claims", "read_account")
49:     def __init__(self, store: Any, audit: Any, week: str,
60:     def read_theme_index(self) -> list[dict[str, Any]]:
66:     def read_theme(self, theme_id: str) -> tuple[dict[str, Any], str]:
77:     def list_run_claims(self) -> list[dict[str, Any]]:
84:     def read_account(self, account_id: str) -> dict[str, Any]:
109:     def _log(self, target: str, detail: dict[str, Any] | None = None) -> None:
```

```
$ grep -n -e ^TOOL_NAMES -e ^    def -e no write tool src/digest/agents/editor/tools.py
```

```
$ grep -n -e ^TOOL_NAMES -e "^    def " -e "no write tool"
src/digest/agents/editor/tools.py
```

```
THEME-0002: Renewal billing omits prior-period credits and does not prorate mid-year upgrades
# Renewal billing omits prior-period credits and does not prorate mid-year upgrades
Members are being billed amounts that ignore what they already paid or are owed.
Evidence:
- Great Lakes Museum Alliance: every renewal statement is missing the balance carried over from the previous
  period, so the invoice total reads far higher than it should; worked around since the spring and now
  noticed by their board.
- Great Lakes Museum Alliance: renewal invoices land without a line for the credit owed from a partial year
  adjustment, and finance chases it every cycle.
- Prairie Land Trust Council: members who upgraded partway through the year are billed the whole annual
  figure again with nothing knocked off, and each invoice is corrected by hand.
Ask: the renewal amount must incorporate carried-over balances, partial-year adjustment credits, and value
  already paid at the time of a mid-year tier change.
## Why it was proposed
Two customers, wrong amounts reaching members, and both are correcting invoices manually every cycle. One
  account reports board-level visibility, which makes it hard to defer.
```

```
$ python -m digest approve --theme THEME-0002 --dry-run --mode replay --repo OWNER/product-feedback
```

```
$ python -m digest approve --theme THEME-0002 --mode replay
--repo jimit1/bi-theme-digest-agent --dry-run
```

Failure behaviour is rejection over repair, with exit codes fixed so a red job is readable without opening the log. A control you can run beats a control you can describe.

Evals

A golden set of 18 assertions, run with one command, and the exit code is the result.

What it asserts

Five must-appear. The two differently worded renewal claims, the two differently worded event check-in claims, and the prospect's course import request, each in the store, cited, on a theme and quoted in the digest.

Two must-not-appear. The private churn risk comment and the employee remark, checked by grep over the whole store, with the commands and exit codes published.

Four personal values. One assertion each, checked with no location excluded, not even the raw fixtures.

Seven structural. Citations resolve, T1 lands on one theme, T2 carries both names in aliases, the prospect ranks below equal-sized customer themes, no personal data anywhere, a rebuild produces the same theme ids, and the stale flag fires.

Run in replay, with zero network calls and zero live model spend, from recordings committed in the store. Anyone can clone the two repositories and reproduce this without being provisioned first.

```
evals/golden_set.yaml via evals/results.md: 5 must-appear, 2 must-not-appear
```

```
T1a [pass] The renewal invoice claim from the Gong call 7782934451002 appears in the store, cited,
          attached to a theme, and quoted in the digest.
          evidence: theme THEME-0002, claim 03a7ca4ae6d8
T1b [pass] The renewal invoice claim from the Salesforce case 5008W00002aQpLrQAK appears in the store,
          cited, attached to a theme, and quoted in the digest.
          evidence: theme THEME-0002, claim 306b8bbab997
T2a [pass] The event check-in claim from the Gong call 7782934451118 appears in the store, cited,
          attached to a theme, and quoted in the digest.
          evidence: theme THEME-0006, claim dcb169d12187
T2b [pass] The attendee kiosk claim from the Gong call 7782934451207 appears in the store, cited,
          attached to a theme, and quoted in the digest.
          evidence: theme THEME-0006, claim ae96fcdf3b0a
T3 [pass] The SCORM upload claim from the Gong call 7782934451311 appears in the store, cited, attached
          to a theme, and quoted in the digest, even though the account is a prospect.
          evidence: theme THEME-0009, claim 61d958ff323f
T4a [pass] The private Salesforce case comment about churn risk never reaches any output.
          evidence: absent from every file in the store
T4c [pass] The Momentive Software account executive line never becomes a client claim in any output.
          evidence: absent from every output; present only in the files that hold what the reader was shown

Result: 18 of 18 assertions passed, exit code 0.
```

```
evals/results.md
```

evals/results.md, per assertion

18 of 18

assertions passed

exit 0

the result is the exit code

Assertion T2b failed in run 1 and run 2 and passes in run 3. The fix was a reader prompt version bump, not a patch to the output.

Drift, measured

The same week was built twice, live, against the same 27 claims. The second opinion was recorded into a scratch copy so it could never overwrite the first.

1.000

claim co-assignment Jaccard: both builds partitioned all 27 claims identically

0.227273

the same overlap once the allocated theme id is included

10 v 10

themes, carrying the same ten scores in the same order

\$0.4954

the second opinion, priced from the rate table

rank	first build	score	second opinion	score
1	THEME-0002 renewal amounts	80	THEME-0005 renewal billing	80
2	THEME-0001 truncated exports	75	THEME-0001 truncated exports	75
3	THEME-0006 offline event check-in	75	THEME-0003 offline event check-in	75

Same claim sets, same scores, same order. Only the four digit ids differ.

The top three looked like it changed, and it did not

Theme ids are handed out in the editor's placeholder order, and week 37 is a cold start into an empty index, so the id measures the labelling rather than the ranking. The flag now compares the claim set each of the top three carries and reads true. The old id comparison is reported beside it and still reads false.

docs/metrics/cost_table.md, section 7

Publishing both is the honest presentation. A metric that flips on a relabelling was worth finding. A metric that keeps flipping after you know why is noise. The first live run of the whole pipeline agreed with itself at 0.792, and that is published too.

Cost

Measured, not estimated. Every figure comes out of the committed audit logs, and a script in the repository recomputes all of it from the store and nothing else.

Week 2026-W37, as run. Extract, 25 calls at the extraction tier, \$1.052313. Edit, 2 calls at the synthesis tier, \$1.218660. Total \$2.270972.

The same week on the frontier model everywhere. \$6.480224, a 2.8535 times bill, for work the golden set says the cheap tier already does correctly. Routing saves \$4.209252 a digest week.

At ten times the volume. Extraction is 92.75 percent of the tokens and 46.3 percent of the cost. The week is \$22.709720 against \$64.802240 frontier everywhere, so the tier map saves \$42.092520 a week, and the saving grows with the column that grows.

The whole store, two ways. \$5.067601 by the published rate table against \$5.755600 by the provider's own accounting, a ratio of 1.1358, because the provider bills a one hour cache write higher than the table does. The provider's figure predates the third analyst question and is carried forward as a declared constant rather than reinvented. Both numbers are labelled everywhere they appear.

```
| stage | tier | model | calls | tokens in | tokens out | cache read | cache write | router table USD |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| extract | extraction | claude-haiku-4-5 | 25 | 532 | 126,559 | 2,389,958 | 143,992 | 1.052313 |
| edit | synthesis | claude-opus-5 | 2 | 6 | 14,926 | 62,784 | 130,254 | 1.218660 |
```

docs/metrics/cost_table.md

docs/metrics/cost_table.md, section 1

Why the cache column is not a mistake

Non-cached input for the whole week is 538 tokens and cache read is 2,452,742. Passing a contract schema writes a large preamble to a one hour cache on the first call of a run and reads it back at a tenth of the input rate for every call after. Almost all input in this pipeline is cached input, by design, and that cheap rate is most of why a week costs two dollars and not twenty.

Rerunning the week with every tier on the small model costs \$0.283140, which is 23.23 percent of the reference build, groups the claims identically at Jaccard 1.000 and passes the same assertions. I still route synthesis to the frontier model, and I publish the number that argues against me.

Agnostic by construction

Application code asks for a tier and never for a model. A model id under the source tree is a build failure, asserted by a test.

1 **Three tiers, one file.** Extraction, synthesis, narrative. config/models.yaml is the only runtime file that names a model, and swapping the map is that one file.

2 **The swap is demonstrable.** One command reruns a week against a different tier map and prints cost, eval pass rate and theme assignment overlap against the reference: \$0.28314, 23.23 percent of reference, and the comparison scores the 16 assertions that bind to week 37 alone, out of the 18 in the golden set, which pass on both sides.

3 **Capability negotiation.** Providers sit behind one adapter interface. The router picks the strongest path a provider declares: native structured output first, then a strict tool call, then the schema in the prompt with validation and one retry in code. The chosen path is in the audit log.

4 **Connectors are the same shape.** A source protocol with two methods. A new source is a new file and a config entry, with no pipeline change.

The router already stepped down a path once during this build, when one contract shape was refused by one transport. It logged which path ran rather than reshaping the contract to suit a transport.

```
cost:   reference $1.2187, alternate $0.2831 (23.2 percent of reference)
eval:   reference 16 of 16 assertions, alternate 16 of 16
overlap: claim co-assignment jaccard 1.000 (with theme ids 0.174), 10 themes against 10, top three CHANGED
        (['THEME-0002', 'THEME-0001', 'THEME-0006'] vs ['THEME-0007', 'THEME-0001', 'THEME-0003'])
```

```
$ make swap-models # python -m digest swap --models config/models.cheap.yaml --week 2026-W37
```

```
$ make swap-models
```

Why the tier map carries no thinking configuration

The thinking interface differs per model family in ways that return an error rather than a warning. One family wants an explicit token budget and rejects an effort setting, another has thinking on by default with an effort scale, a third needs its mode stated explicitly. So the adapter derives it from the family in code and the config file never carries a budget it could not explain.

The build was itself an agent system

The thing being demonstrated is not only the product. This deliverable was built by a parallel fanout of single purpose agents under one orchestrator.

7

waves, numbered 0 to 6, each gated on the wave before it

30

worker tasks, one structured envelope returned per worker

4

model tiers ran the build, and the roster is committed

1

orchestrator, holding the plan and dispatching every brief

The orchestrator dispatched briefs and read envelopes, and opened product files only to triage QA findings.

The wave shape

Wave 0 is the contracts, then the corpus and the connectors, then the pipeline and the agents, then integration and the live runs, then the write up, then a wave of adversarial review that built nothing, then this deck. A worker only starts when the wave it depends on has returned.

What ran, not what was planned

The orchestrator on one model, the writer and reviewer tier on the frontier model, the cheap tier on a mid model. The plan named a different cheap tier and the harness offered a different one, so the committed roster file records what actually ran. The product's own numbers are the proof the shape worked: 55 claims, 0 rejected, 18 of 18 assertions, 531 tests, byte identical replay.

What stops a parallel build from rotting

Four rules, written down before any fanout. Context rot is the failure mode of a large agent build, and a bigger context window is not the fix.

1 A contract pack first

Every schema was written and frozen before a single worker started, so no worker ever had to guess what another worker would return.

2 Exclusive file ownership

Every worker owns a set of paths and writes nowhere else, so two agents never race on one file and a merge conflict is a planning bug rather than a runtime one.

3 Self-contained briefs

A worker gets only the contract slice it needs and never talks to another worker, so a brief is testable on its own and a failed worker can be re-run without replaying the build.

4 Envelopes with an assumptions array

Anything a worker inferred rather than being told goes in a named array rather than buried in code. That array is how the repository's decision log came to exist at all.

The same rule runs inside the product. No agent returns prose to another agent: every model call returns a named, versioned schema from the contract pack, validated in code, retried once with the validation error appended, then rejected and logged. Never patched, never coerced. That single decision is what keeps a multi agent pipeline from becoming a parsing problem.

The QA wave that built nothing

One whole wave of agents wrote no product code. Their only job was to try to break what the previous waves had shipped.

The citation auditor

It resolved claims by hand, opening the source files itself and checking the quoted text, without using the pipeline's own verifier. The reason: a verifier that is wrong in the same direction as the extractor is invisible to itself.

It also found a real bug

The digest reported ten personal data redactions where five values were planted, because the scrubber counted the turn text and the per sentence copies of it. The redaction was right and the count was doubled. The fix and the store regeneration are in the repository.

30 / 30	8	0
sampled claims resolved	trap claims, all covered	unresolved
55 / 55	125	0
full sweep of claims	digest evidence lines	bad lines

A compliance sweep. Both repositories, eight categories: dash characters, non-ASCII, secrets, stray real email addresses, URL-shaped text, lowercase company name and voice drift all came back with zero real findings, and zero model ids under the source, connector and tools trees.

A portability agent. It cloned the repository as a stranger and found a real defect: the weekly build was not idempotent, so a clone of the published store rebuilt weeks it already had. Fixed at the root, with a test.

Acceptance, re-audited after the portability fix: 21 of the 24 acceptance lines pass, 0 fail, and 3 are not testable on this machine. Before the fix it read 17 pass and 3 fail, on lines A1, A7, A15.

In production

The same pipeline, pointed at the real APIs, on a schedule, with nobody watching it.

- 1 Nightly ingestion off a watermark.** The store carries a last ingest run and a last ingest day, and each night pulls only what is newer. Nothing is fetched twice, and a skipped night shows as a thin run rather than as silence. 15 nightly ingests produced 55 claims from 53 source documents in the sample.
- 2 Connectors swapped for the real APIs.** The mock already serves the real response shapes, cursor pagination included, so the change is a base URL and a credential with no translation layer to write.
- 3 Scoped read-only credentials.** One read-only credential per source, and one identity per agent rather than one shared service account, because an audit log that says the integration user did it answers nothing. Secrets in a vault, read at start up, never on disk.
- 4 The approval gate stays.** The agent proposes and a person files. It earns more room against a measured acceptance rate, never against time served.

```
name: ingest

on:
  schedule:
    - cron: "0 6 * * *"
```

`.github/workflows/ingest.yml`

`.github/workflows/ingest.yml`

On Bedrock, in two sentences

I would run this on the Claude platform or the API directly, because the constraint here is speed of adoption and the platform's own identity and audit already satisfy the requirement. I would reach for Bedrock when the constraint is data residency in a named region, identity native to the cloud account for agents already running inside it, private networking with no public egress, or consolidated billing against committed spend.

The router exists so that is a one file change. What actually differs between the platforms is model id translation.

Two workflows are committed and both run green with no secret configured, because they run in replay against the committed corpus. That is deliberate and it is stated as a comment in the workflow file.

Future builds

Nine items are written down, ordered, each with what it needs to be safe. None of it is built, and saying so is the point. Here are the first four.

1 A golden set that grows itself

From human corrections, rejected claims and production incidents, versioned in the store so the eval history is as auditable as the digest history.

Needs a correction mechanism on the digest surface first. Without somewhere for a human to say this one is wrong, there are no corrections to learn from.

2 An adaptive eval agent, gated

It reads the run logs and the corrections, finds the intents whose pass rate is sliding, and opens a pull request. It does not merge.

Needs per intent scoring. An agent that can only see an aggregate pass rate cannot tell you what to change, so it will guess.

3 Trust tiers on the filing gate

Propose only, then draft into an approval queue a human signs off, then file directly within a named scope and a named budget.

Needs proposal acceptance tracked per theme type, so widening is a number someone can point at. Widen on evidence, never on time served.

4 Drift detection as a scheduled job

Per intent pass rates on a schedule with an alert when one slides, and the alert fires on thin runs rather than only on failed ones.

Needs a recorded expectation of volume per source per day. Without it a genuinely quiet week and a broken connector look identical.

The other five are in the same document, in the same shape: retrieval when the index outgrows layer one, more sources against the same connector protocol, batching and prefix caching before any model downgrade, a feedback surface for the product team, and the approval queue as a real surface.

What I cut, and why

Every line below is a decision with a reason attached, which is the same standard I held the agent to.

- **Real Gong and Salesforce API integration.**

The brief said mock, and the connector boundary is the seam that makes the swap a credential change.

- **Authentication, multi-tenancy and role based access on the store.**

Git permissions are the prototype's access control, and I would not ship that.

- **Vector retrieval.**

At this corpus size the theme index is smaller and more auditable than an embedding store. When it stops fitting comfortably in layer one, retrieval goes behind a tool with citations required.

- **A web interface or a hosted service.**

The digest is a file and a page, because that is what a product team actually reads.

- **The adaptive eval agent that updates other agents' context.**

It is the right next stage and it belongs behind the same gate, so it is designed in the document and not built.

- **Chat delivery into Slack or Teams.**

One connector, no new thinking, not worth the hours.

Jim Mehta, 2026-09-16. Two public repositories: the code, and the context store it writes to.